

Robustez bajo carga

Escenario de prueba 1 – Alto número de serpientes

Configuración

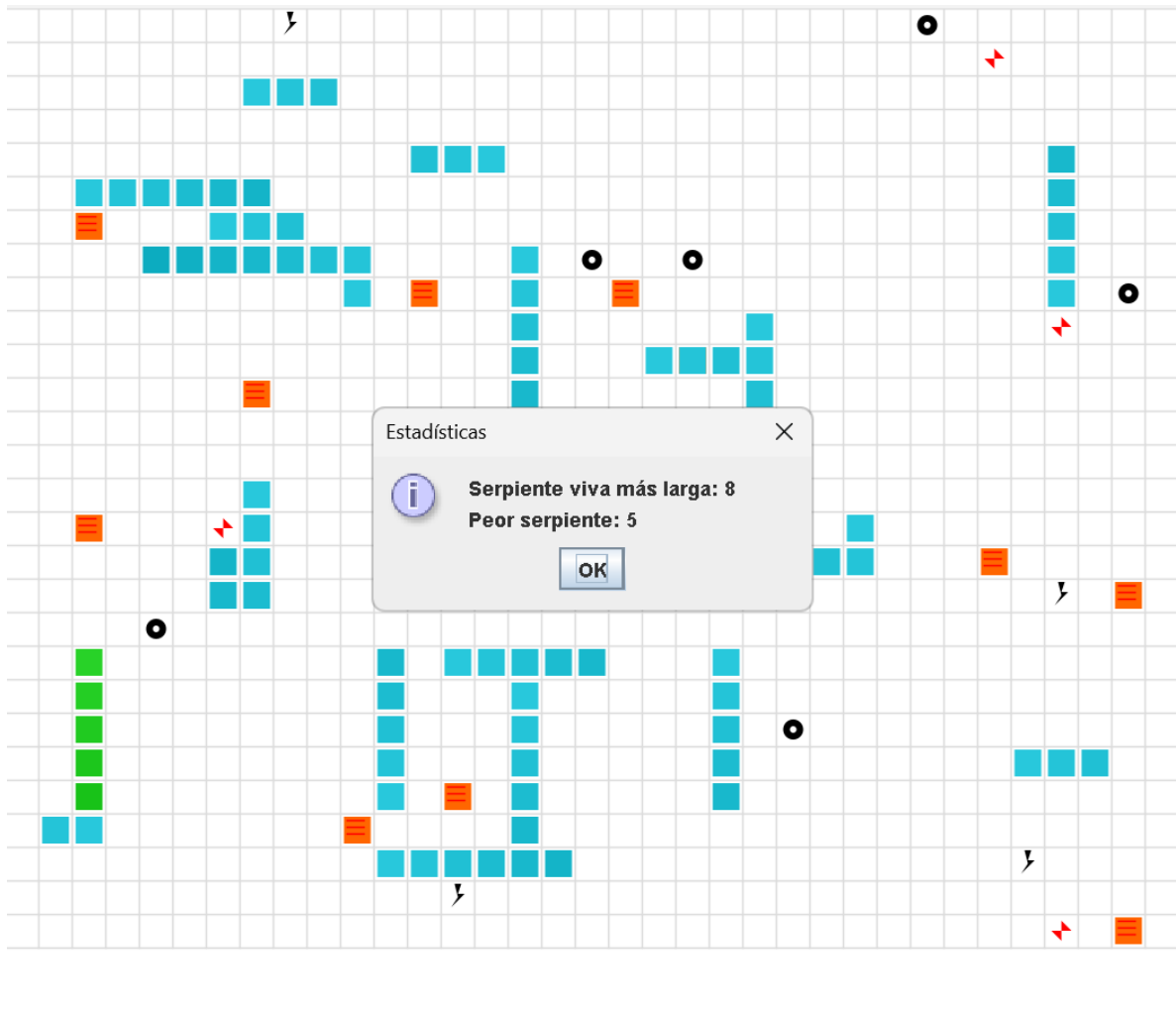
- Ejecución con:
- `java -Dsnakes=20`
- Velocidad base sin modificar.

Resultado observado

- No se presentan `ConcurrentModificationException`.
- No hay bloqueos ni congelamientos de la interfaz.
- Las serpientes se mueven y mueren de forma independiente.
- La mayoría de serpientes son azules.

Justificación técnica

- Las estructuras compartidas del tablero se acceden dentro de regiones sincronizadas.
- La interfaz gráfica trabaja sobre copias defensivas del estado del tablero.
- Cada serpiente se ejecuta en su propio hilo sin compartir estado mutable no protegido.



Escenario de prueba 2 – Aumento de velocidad de ejecución

Configuración

- Reducción de tiempos:
 - `baseSleepMs` = 20
 - `turboSleepMs` = 5

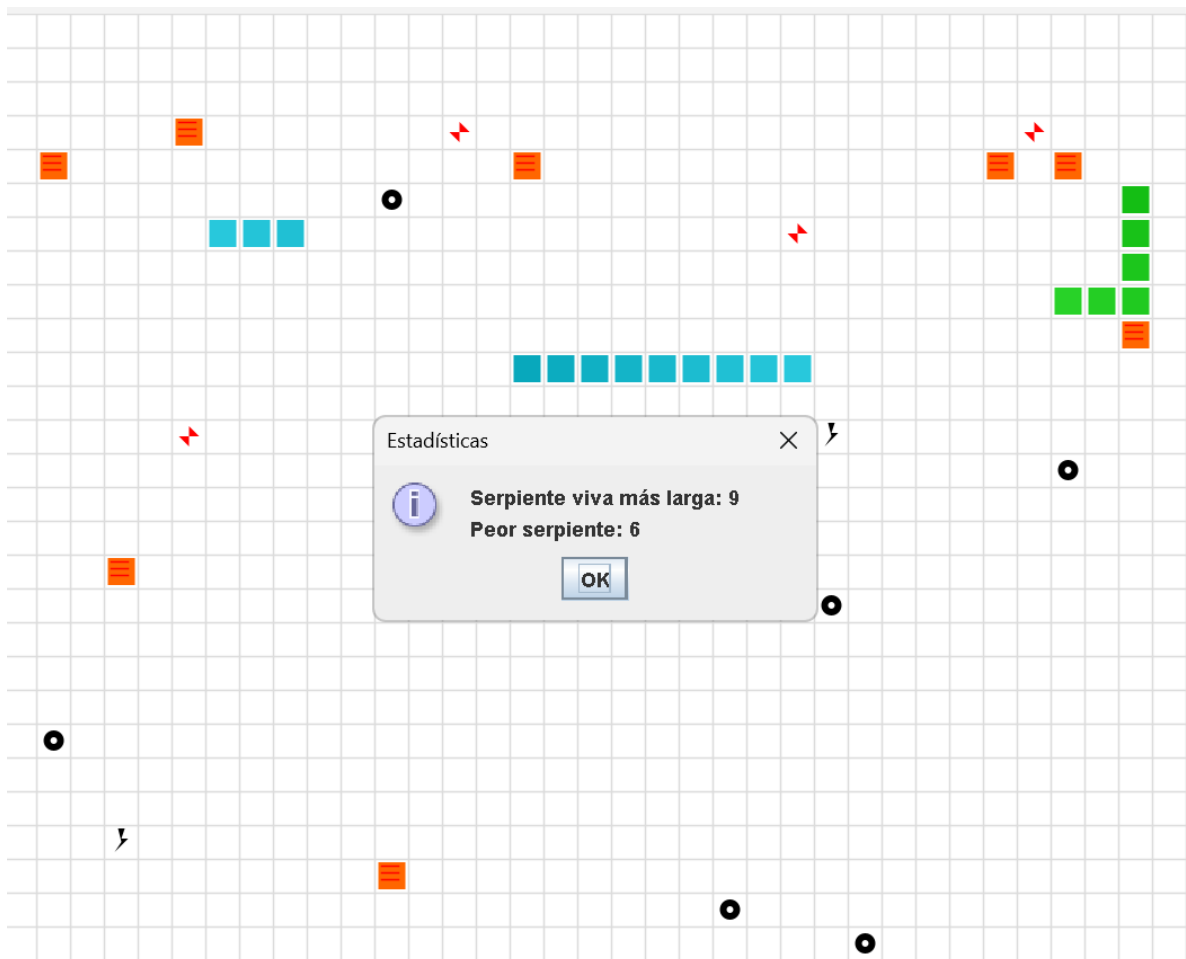
Resultado observado

- No aparecen lecturas inconsistentes del estado del tablero.
- El renderizado no muestra estados intermedios (“tearing”).
- Las serpientes siguen respetando colisiones y crecimiento.

Justificación técnica

- El avance del juego ocurre dentro de regiones críticas bien definidas.

- El método `snapshot()` desacopla la UI de los hilos de simulación.
- No se utiliza espera activa; los hilos liberan CPU correctamente.



Escenario de prueba 3 – Pausa agresiva (UI + concurrencia)

Configuración

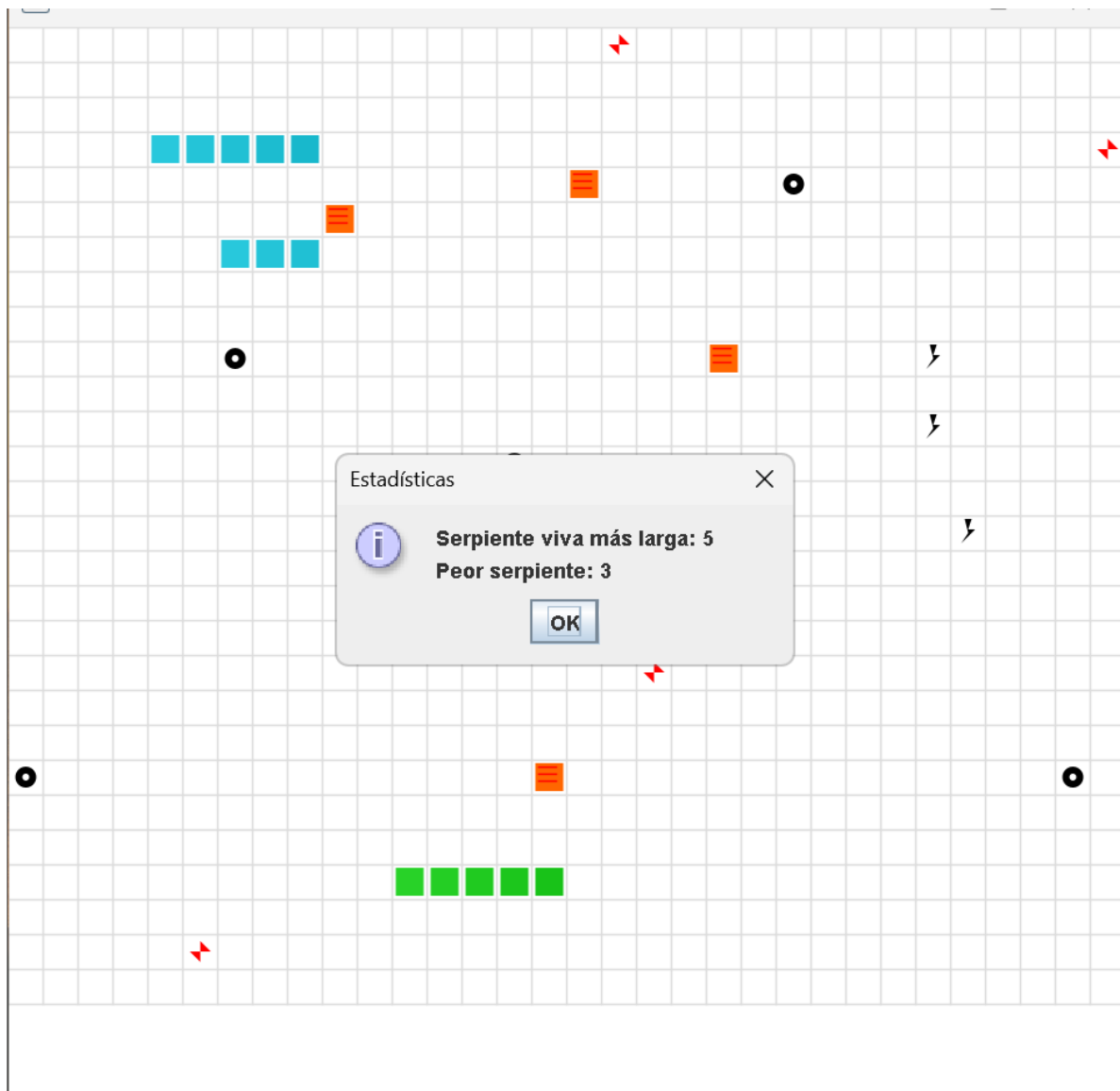
Mientras el juego corre:

- Presiona **SPACE** repetidamente
- Clic rápido en **Action / Resume**
- Hazlo cuando hay muchas serpientes

Resultado observado

- No hay deadlock
- Todas las serpientes se detienen juntas

- Al reanudar, continúan normalmente



Escenario de prueba 4 – teleports + turbo bajo carga

Configuración

En Board constructor:

- `createTeleportPairs(6);` // antes 2
- `for (int i = 0; i < 8; i++) turbo.add(randomEmpty());` // antes 3

Resultado observado

- No hay saltos inconsistentes
- No teletransporta a posiciones inválidas

- El turbo no se queda pegado

